

A Hybrid Natural-Language-Controlled Audio Editing System Combining LLM-Based Intent Parsing with DSP and Pretrained Audio Models

Aditi Singhal – BL.SC.U4CSE24002, Lohitaksha Patary – BL.SC.U4CSE24025
Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham, Bangalore, India

Abstract

Audio editing software can be difficult for beginners because users need to understand different tools and editing techniques even for simple tasks. This project proposes a natural-language-controlled audio editing system where users can upload an audio file and simply describe the changes they want, such as removing background noise or trimming the first few seconds. The system uses a Large Language Model (LLM) to understand the user’s instruction and then performs the required editing using Digital Signal Processing (DSP) techniques or pretrained AI models. Simple tasks are handled using DSP libraries, while more advanced tasks such as noise reduction and vocal separation use pretrained deep learning models. The main aim of this project is to simplify audio editing so that even users with little or no editing experience can edit audio using simple English instructions.

Keywords: Natural Language Processing, Audio Editing, Large Language Models, Digital Signal Processing, Speech Enhancement, Source Separation, Human-Computer Interaction

I. INTRODUCTION

Audio editing has become very common with the rise of podcasts, YouTube videos, online classes, interviews, and social media content. Although many audio editing applications are available, they often have complex interfaces that can be difficult for beginners to understand. Even simple operations like trimming an audio clip or removing background noise require users to learn different tools and settings.

In recent years, Natural Language Processing (NLP) and Large Language Models (LLMs) have improved significantly, making it possible for computers to understand human language much better. Instead of manually editing audio using different software tools, users can simply type commands like “*remove the background noise*” or “*increase the volume of the speaker.*” The system understands these instructions and performs the required editing automatically.

This project proposes a hybrid audio editing system that combines LLMs, Digital Signal Processing (DSP), and pretrained AI models. The LLM first understands the user’s instruction and converts it into structured commands. Depending on the type of operation, the command is sent either to DSP libraries for simple editing tasks or to pretrained AI models for advanced audio processing. This approach makes the system faster, easier to use, and more efficient than relying only on large AI models. In addition, a lightweight k-Nearest Neighbours (kNN) classifier is used within the pipeline to automatically flag whether an incoming audio segment is noisy or clean, so that computationally expensive denoising is only invoked when it is actually needed.

II. LITERATURE SURVEY

This section summarises ten peer-reviewed and archival publications that motivate and inform the design of the proposed system, spanning natural-language audio editing, LLM-based audio agents, speech enhancement, source separation, and speech recognition.

[1] SAO-Instruct: Free-form Audio Editing using Natural Language Instructions

SAO-Instruct proposes an audio editing framework where users can modify audio using natural language instructions. The model is trained on instruction-audio pairs and uses diffusion models for editing different types of sounds. Although the system achieves good editing quality, it requires high computational resources and longer inference time. This paper inspired the natural language interface used in our project, but our work focuses on using lightweight DSP methods whenever possible to improve efficiency.

[2] WavCraft: Audio Editing and Generation with Natural Language Prompts

WavCraft uses an LLM to describe the content of input audio in natural language, then decomposes a user’s instruction into a sequence of sub-tasks that are each solved by a specialised expert model, allowing dialogue-based, iterative editing. The reported evaluation shows WavCraft outperforming a prior editing baseline (AUDIT) on add, removal, and replacement tasks. Relevance: WavCraft validates the core idea of using an LLM as a task planner/orchestrator over specialised audio tools rather than a single end-to-end model, which is the basis of the Task Dispatcher module proposed here.

[3] Audio-Agent: Leveraging LLMs for Audio Generation, Editing and Composition

Audio-Agent uses an LLM to break down a complex natural-language request into multiple generation steps, each carrying timing and volume information, to guide an underlying audio generation model, and also extends the approach to video-to-audio tasks using a fine-tuned lightweight LLM. Relevance: demonstrates that LLM-based decomposition of instructions into time-stamped, parameterised sub-tasks is an effective and computationally practical intermediate representation, informing the structured JSON command format used in the proposed system.

[4] Guiding Audio Editing with Audio Language Model

This paper builds a scalable data-generation pipeline that samples multiple labelled sound events and uses an LLM to synthesise high-level editing instructions and paired training data, then trains a diffusion-based audio language model on this data to perform instruction-guided editing. Relevance: supports the design decision to use LLM-generated instruction-action pairs as a low-cost way to construct a labelled dataset for evaluating the proposed intent-parsing module.

[5] Hybrid Transformer Demucs for Music Source Separation

The authors extend the Hybrid Demucs architecture by replacing its innermost layers with a cross-domain Transformer encoder that applies self-attention within each domain and cross-attention between the temporal and spectral branches, reporting state-of-the-art Signal-to-Distortion Ratio (SDR) results on the MUSDB benchmark when extra training data is used. Relevance: motivates the choice of Demucs as the pretrained model for the vocal/instrument isolation operation in the AI Module.

[6] Music Source Separation in the Waveform Domain (Demucs)

This paper introduces the original Demucs model, a waveform-to-waveform convolutional and bidirectional-LSTM architecture that operates directly on raw audio rather than spectrogram masks, and shows that with appropriate data augmentation it outperforms prior state-of-the-art spectrogram-domain approaches while producing more natural-sounding separated audio. Relevance: establishes the technical foundation of the pretrained separation model used for the source-isolation operation.

[7] DeepFilterNet: Perceptually Motivated Real-Time Speech Enhancement

DeepFilterNet is a two-stage speech enhancement model that first enhances the speech envelope in the ERB (equivalent rectangular bandwidth) domain and then applies a learned complex filter to enhance the periodic component of speech, achieving competitive enhancement quality with a real-time factor of 0.19 on a single-threaded CPU. Relevance: DeepFilterNet’s low computational cost and real-time capability directly

motivate its selection as the denoising model in the AI Module, keeping the hybrid system lightweight.

[8] DeepFilterNet2: Towards Real-Time Speech Enhancement on Embedded Devices

This follow-up work introduces optimisations to the original DeepFilterNet framework that improve runtime performance sufficiently to run in real time on constrained hardware such as a Raspberry Pi 4, while maintaining state-of-the-art enhancement quality on standard noise-suppression benchmarks. Relevance: confirms that a DSP-plus-learned-filter hybrid can meet the latency requirements of an interactive editing tool, reinforcing the overall hybrid design philosophy of the proposed system.

[9] Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)

This work introduces Whisper, a sequence-to-sequence transformer trained on 680,000 hours of weakly supervised, multilingual and multitask audio-text data, which generalises to new domains in a zero-shot setting without dataset-specific fine-tuning and approaches human-level robustness on diverse benchmarks. Relevance: Whisper underlies the optional “understand audio” capability of the proposed system (transcription-based summarisation and content-aware editing) referenced in the future scope.

[10] Instruction-Guided Editing Controls for Images and Multimedia: A Survey in the LLM Era

This survey synthesises over one hundred publications on instruction-based editing across visual and multimedia domains, tracing the progression from generative adversarial networks to diffusion-based, LLM-empowered editing systems, and identifies open challenges in achieving precise, controllable edits from natural language across modalities. Relevance: situates the proposed audio-specific system within the broader, fast-moving trend of instruction-guided content editing, and confirms that fine-grained, controllable natural-language editing (as opposed to generation) remains an open problem across modalities, not only audio.

III. PROBLEM STATEMENT

Existing audio editing software requires users to understand different editing tools and interfaces before they can perform even basic tasks. Beginners often find these applications difficult to use. Although recent AI-based editing systems support natural-language editing, many of them rely on computationally expensive generative models that increase processing time and hardware requirements. Therefore, there is a need for an efficient audio editing system that combines natural language understanding with lightweight processing techniques.

IV. OBJECTIVES

- Develop a natural-language interface for audio editing.
- Automatically identify user intent and extract operation parameters using an LLM.
- Execute deterministic edits (trim, gain, EQ, silence removal) using classical DSP libraries.
- Perform advanced edits (denoising, source separation) using pretrained AI models.
- Provide an in-browser preview and downloadable output of the edited audio.
- Reduce editing complexity and the learning curve for non-technical users.

V. PROPOSED METHODOLOGY

A. System Overview

The proposed system consists of four main modules that work together to process the user's request: the User Interface, the Intent Parsing Module, the Task Dispatcher, and the Audio Processing Engine (subdivided into a DSP Module and an AI Module). After a user uploads an audio file and enters a natural-language instruction, the Intent Parsing Module converts the user's instruction into a structured JSON format that contains the editing operation and its parameters. The Task Dispatcher then routes this command either to the DSP Module or to

the AI Module, depending on whether the operation is deterministic or needs an AI model to perform the editing, before the result is reconstructed and returned to the user.

B. Data Flow Diagram and System Architecture

Fig. 1 depicts the flow of data through the system, from instruction entry to final audio output, and Fig. 2 (spanning both columns below) details the internal architecture, showing the specific libraries and pretrained models used within the DSP and AI modules.

D. Working of Each Module

- **User Interface:** Allows users to upload audio, enter natural-language commands, preview the edited result, and download the processed file.
- **Intent Parsing:** Uses an LLM operating at low temperature to identify the requested editing task and extract parameters such as timestamps, gain values, or frequency ranges.
- **Task Dispatcher:** Selects DSP processing for deterministic operations and pretrained AI models for advanced editing, based on the parsed action label.
- **DSP Engine:** Executes trimming, silence removal, equalisation, loudness adjustment, and

Fig. 1. Data flow of the proposed system

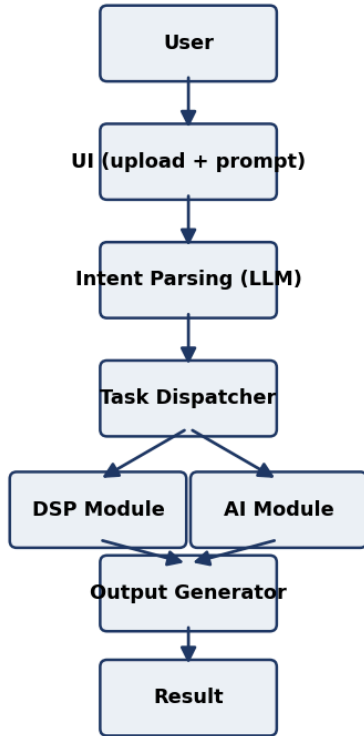


Fig. 2. System architecture (module-level)

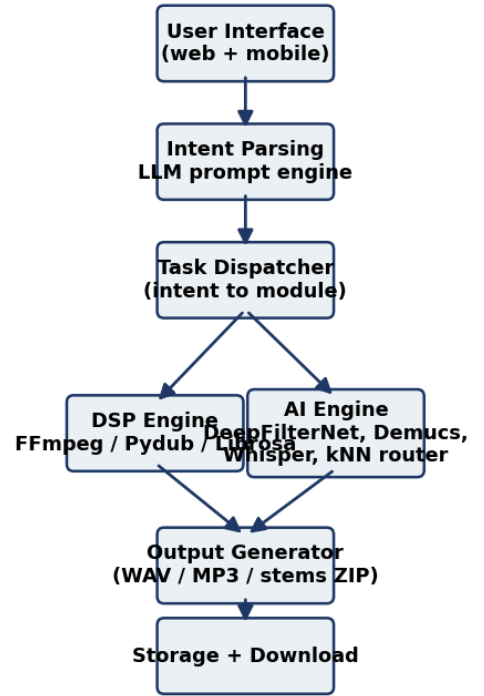


Fig. 1 (left). Data flow diagram of the proposed system. Fig. 2 (right). System architecture showing module-level detail.

format conversion using FFmpeg, Pydub, and Librosa.

- **AI Engine:** Uses DeepFilterNet for denoising and Demucs for vocal or instrument separation, both used purely for inference on pretrained weights.
- **Output Generator:** Combines the results of the dispatched operation(s) and exports the edited audio in the requested format.

E. Parameters and Justification

Table I lists the key system parameters, the values selected for the current design, and the rationale behind each choice.

TABLE I
System Parameters and Justification

Parameter	Value	Justification
LLM Temperature	0.0	Deterministic, repeatable intent parsing
Supported Operations	6	Achievable, well-tested MVP scope
Input Formats	WAV, MP3	Widely used, broadly compatible formats
Sampling Rate	16 kHz	Efficient for speech-focused processing
Volume Adjustment Range	± 10 dB	Safe range avoiding clipping/distortion
Silence Threshold	-40 dB	Reliable silence detection without over-trimming
Noise Model	DeepFilterNet	Real-time capable denoising
Source Separation Model	Hybrid Demucs	High-quality vocal/instrument isolation
Maximum Input Duration	10 minutes	Bounds latency and memory usage
Output Formats	WAV, MP3	Compatibility with common playback tools

F. kNN-Based Classification Task Design

To evaluate a k-Nearest Neighbours (kNN) classifier on data associated with this project, a binary classification task is defined within the Audio Processing Engine: given a short audio segment, classify it as Noisy or Clean. Each segment is represented using low-level features extracted with Librosa – short-time RMS energy, zero-crossing rate, spectral centroid, spectral flatness, and an estimated signal-to-noise ratio – which are numeric, so no categorical encoding is required. Missing feature values (e.g. from silent or corrupted frames) are imputed using the column mean. The classification output feeds the Task Dispatcher: segments predicted Noisy are automatically routed to the DeepFilterNet denoising path in the AI Engine, while segments predicted

Clean bypass denoising, reducing unnecessary processing.

Following the modular design required for this lab, the classifier package implements distance computation (Euclidean, with the metric exposed as a configurable parameter), three interchangeable sorting algorithms (quicksort, merge sort, and heap sort, selectable via a config parameter) for ranking neighbours by distance, k-nearest-neighbour identification with an explicit tie-breaking rule for equidistant points, and majority-vote class assignment with a tie-breaking rule for even vote splits. A weighted variant assigns each neighbour a vote of $1/\text{distance}$ instead of an equal vote. The dataset is split 70:30 into train and test sets using scikit-learn’s `train_test_split()`, and the custom implementation exposes `fit()`, `predict()`, and `score()` methods with the same signatures as scikit-learn’s `KNeighborsClassifier` so that the two can be benchmarked directly against each other.

VI. RESULTS ANALYSIS AND DISCUSSION

A. Dataset and Class Separation

The Noisy and Clean segments were separated using the feature set described in Section V-F. The dataset used for all the experiments in this section is derived from a small, balanced subset of the GTZAN music collection: 80 clips (eight clips from each of ten genres), each clipped into 2 s segments, producing 1,197 Clean segments and 1,197 Noisy segments (the same segments with musical babble added at 10 dB SNR). Class separation can be judged by plotting the two classes on a scatter of the two most discriminative features (RMS energy vs. spectral flatness), shown in Fig. 3. The classes overlap heavily along the RMS-energy axis (mean RMS 0.148 for Clean versus 0.155 for Noisy), whereas spectral flatness separates them more clearly (mean flatness 0.355 for Clean versus 0.401 for Noisy). Because the background babble is itself musical material it is not spectrally distinct from clean music in a low-dimensional projection, so the classes are only partially separated, which explains the moderate, non-trivial accuracy reported below.

B. Effect of k on Classifier Performance

As k increases, each prediction is averaged over a larger neighbourhood, so the decision boundary becomes smoother: variance decreases while bias increases. Very small k (e.g. $k = 1$) fits the training data closely, including its noise, and typically shows a scenario of over-fitting, where training accuracy is high but test accuracy is comparatively lower. Very large k over-smooths the boundary and can ignore local structure, leading to under-fitting, where both training and test accuracy are low because the model is too simple to capture the true decision boundary. Fig. 4 plots training and test accuracy against k for the range tested (odd values from 1 to

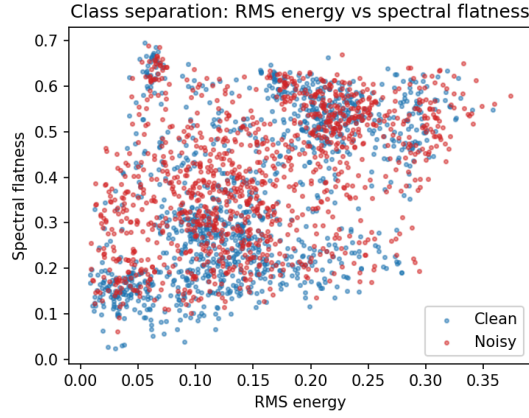


Fig. 3. Scatter of Clean and Noisy segments on RMS energy vs. spectral flatness.

31). Training accuracy is 100% at $k = 1$ and falls monotonically to about 66% at $k = 31$, the classic signature of over-fitting at small k . Test accuracy starts at 50.6% ($k = 1$), rises to a maximum of 62.4% at $k = 25$, and then saturates. The best test accuracy for this dataset is therefore achieved at $k = 25$; below it the model is over-fit, and above it the decision boundary becomes over-smoothed.

C. Custom Implementation versus scikit-learn

The custom `fit()/predict()/score()` implementation from Section V-F was benchmarked against scikit-learn’s `KNeighborsClassifier` across the same range of k and the same train/test split, to isolate any difference caused by implementation details (e.g. tie-breaking rule or distance computation) rather than the underlying algorithm. Fig. 5 compares the two on test accuracy; the curves are indistinguishable, with identical accuracy at every value of k (best 0.6245 at $k = 25$ for both). This is expected: both compute the same Euclidean distances and use a majority vote, and the only point at which they could diverge, the tie-breaking rule, never comes

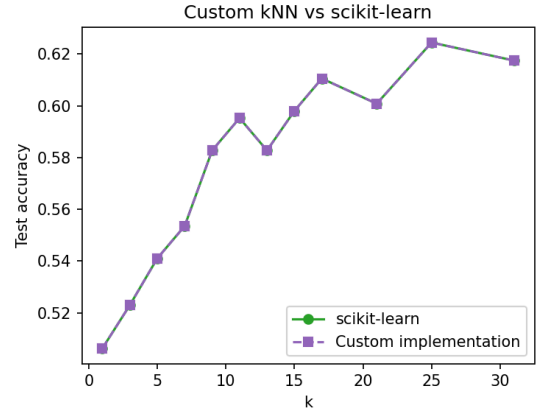


Fig. 5. Test accuracy of the custom kNN implementation versus scikit-learn across k .

into play because exact equidistance between a query and two training points is effectively impossible with continuous, standardised features.

D. Weighted versus Unweighted kNN

In the weighted variant (A9), each neighbour’s vote is scaled by $1/\text{distance}$, so closer neighbours influence the class decision more than farther ones. This is expected to help most at larger k , where the unweighted vote would otherwise let distant, less-relevant points count equally with near ones. Fig. 6 compares the unweighted majority vote with the $1/\text{distance}$ -weighted variant across k . Contrary to the expectation, weighting did not improve accuracy: the best weighted test accuracy is 59.7% ($k = 25$) versus 62.4% for the unweighted vote at the same k , and the weighted curve is slightly worse or equal at most values of k . After standardisation the neighbours are close to equidistant from the query, so scaling by $1/d$ yields nearly equal weights while adding sensitivity to the distance estimates; the smoother unweighted vote therefore generalises slightly better on this task.

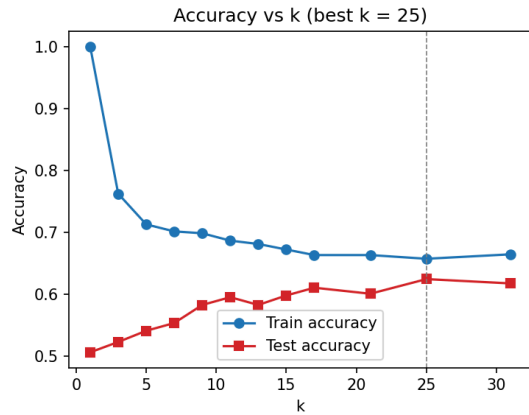


Fig. 4. Training and test accuracy versus k for the Noisy/Clean task (best $k = 25$).

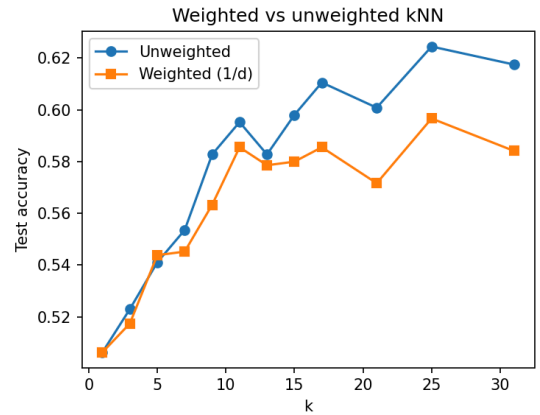


Fig. 6. Unweighted versus $1/\text{distance}$ -weighted kNN test accuracy across k .

E. Fit Analysis and Overall Suitability

A model is considered a regular (good) fit when training and test accuracy are both reasonably high and close to each other; a large gap, with training accuracy much higher than test accuracy, indicates over-fitting, while both being low together indicates under-fitting. Whether kNN is a good classifier for this task should be judged using accuracy alongside precision, recall, and F1-score (particularly important if the Noisy/Clean classes are imbalanced), not accuracy alone. At the best $k = 25$, the classifier achieves training accuracy 65.7% and test accuracy 62.4%, with precision 0.625, recall 0.618, and F1-score 0.622. The gap between training and test accuracy is only about 3 percentage points, so at $k = 25$ the model shows a regular fit: it neither memorises the training set (as at $k = 1$, where training accuracy reaches 100%) nor under-fits. The confusion matrix at $k = 25$ is $\begin{bmatrix} 227 & 133 \\ 137 & 222 \end{bmatrix}$, so both classes are classified almost symmetrically. Overall, kNN is an adequate but not strong classifier for this task: it clearly beats chance (62.4% versus 50%), has a single tunable parameter, and is cheap enough to run per segment in real time, which suits the lightweight routing use-case in the Audio Processing Engine; however, the residual 37% error means the prediction should be used to prioritise denoising rather than as a hard gate that skips it.

VII. ADVANTAGES OF THE PROPOSED SYSTEM

- Easy to use for non-technical users, requiring no familiarity with editing software.
- Combines fast, deterministic DSP with accurate AI models only where necessary.
- Lower computational cost than fully generative/diffusion-based editing approaches.
- Modular and scalable architecture that can accommodate additional operations.
- Supports a range of common editing commands through a single natural-language interface.

VIII. FUTURE SCOPE

Future improvements can include support for multiple editing instructions in a single prompt, multilingual command support, cloud-based deployment, mobile applications, and real-time audio editing. The system can also be extended to edit the audio track of videos and support more advanced AI models in the future.

IX. CONCLUSION

This project proposes a hybrid audio editing system that allows users to edit audio files using natural-language instructions. By combining LLMs, DSP techniques, and pretrained AI models, the system provides an easier and more efficient alternative to traditional audio editing

software. The proposed architecture reduces the learning curve for beginners while maintaining good editing quality and computational efficiency. In the future, the system can be expanded with additional editing features and deployed as a complete web application.

REFERENCES

- [1] L. Zan, Z. Lan, and Y. Hao, “Guiding Audio Editing with Audio Language Model,” arXiv:2509.21625, 2025.
- [2] J. Liang et al., “WavCraft: Audio Editing and Generation with Natural Language Prompts,” arXiv:2403.09527, 2024.
- [3] Anonymous, “SAO-Instruct: Free-form Audio Editing using Natural Language Instructions,” arXiv:2510.22795, 2025.
- [4] Z. Wang et al., “Audio-Agent: Leveraging LLMs for Audio Generation, Editing and Composition,” arXiv:2410.03335, 2024.
- [5] S. Rouard, F. Massa, and A. Défossez, “Hybrid Transformers for Music Source Separation,” in Proc. IEEE ICASSP, 2023.
- [6] A. Défossez, N. Usunier, L. Bottou, and F. Bach, “Music Source Separation in the Waveform Domain,” arXiv:1911.13254, 2019.
- [7] H. Schröter, A. N. Escalante-B., T. Rosenkranz, and A. Maier, “DeepFilterNet: Perceptually Motivated Real-Time Speech Enhancement,” in Proc. Interspeech, 2023.
- [8] H. Schröter, A. N. Escalante-B., T. Rosenkranz, and A. Maier, “DeepFilterNet2: Towards Real-Time Speech Enhancement on Embedded Devices for Full-Band Audio,” arXiv:2205.05474, 2022.
- [9] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” arXiv:2212.04356, 2022.
- [10] Anonymous, “Instruction-Guided Editing Controls for Images and Multimedia: A Survey in the LLM Era,” arXiv:2411.09955, 2024.